

# Computer Graphics — Structured Notes

---

## Chapter 1: Introduction

### Object Space vs. Image Space

| Space               | Description                                                          |
|---------------------|----------------------------------------------------------------------|
| <b>Object Space</b> | Continuous abstract space where shapes are defined using coordinates |
| <b>Image Space</b>  | Discrete pixel grid (display monitor) where the image is produced    |

### Key Areas

- **Geometric Representation** — defining objects using coordinates (vertices, etc.)
- **Transformation** — mapping objects to image space using matrices (controls position, orientation, size)
- **Scan Conversion** — converting continuous figures to discrete pixels (rasterization)
- **Anti-aliasing** — reducing the "staircase" distortion caused by scan conversion

### 2D Graphics Pipeline

$$\text{Object Coordinates} \rightarrow \text{World Coordinates} \rightarrow \text{Device Coordinates}$$

### 3D Graphics Concepts

- **Projection** — mapping 3D objects onto a 2D display surface
- **Hidden Surface Removal** — hiding surfaces obscured from the viewer
- **Illumination & Shading** — simulating light reflection for realism
  - *Local illumination* — direct light only
  - *Global illumination* — includes reflected light and transparency

### Allied Fields

- **Image Processing** — pixel-based operations on existing images (not synthesizing from scratch)
  - **Computer-Human Interaction** — interfaces and logical input devices
- 

## Chapter 2: Image Representation

### Digital Image Basics

| Term              | Definition                                               |
|-------------------|----------------------------------------------------------|
| <b>Resolution</b> | Number of pixels per unit length (e.g., pixels per inch) |

| Term                | Definition                                |
|---------------------|-------------------------------------------|
| <b>Aspect Ratio</b> | Width ÷ Height (e.g., 1920 × 1080 → 16:9) |

## Color Models

### RGB (Additive — for displays)

$$\text{Color} = (R, G, B), \quad R, G, B \in [0, 1]$$

- Black = (0,0,0), White = (1,1,1)
- The **gray axis** is the diagonal from black to white

### CMY (Subtractive — for printers)

$$C = 1-R, \quad M = 1-G, \quad Y = 1-B$$

## Image Representation Methods

### Direct Coding

Each pixel stores actual color values.

| Type             | Bits/pixel | Colors        |
|------------------|------------|---------------|
| Binary (B&W)     | 1          | 2             |
| Grayscale        | 8          | 256           |
| True color (RGB) | 24         | ~16.7 million |

### Lookup Table (Color Map)

Each pixel stores an **index** into a color table.

- 8-bit pixel → 256-entry table, each entry = 24-bit RGB
- **Memory formula:**  $\text{Entries} \times 3 \text{ bytes}$ 
  - Example: 2-byte pixel → 65,536 entries →  $65,536 \times 3 = \mathbf{196,608 \text{ bytes}}$

## Output Devices

### CRT Monitors

- Electron guns strike phosphor-coated screen → emits light
- Requires refresh (~60 Hz) to avoid flicker
- Uses 3 guns (R, G, B) with shadow masks

### Printers (CMYK)

- Paper is white; ink **subtracts** light
- Black ink used separately (cheaper, better quality than mixing C+M+Y)

#### Techniques to simulate shades:

| Technique              | Method                                | Resolution |
|------------------------|---------------------------------------|------------|
| <b>Halftoning</b>      | Varies dot size in a grid             | Reduced    |
| <b>Dithering</b>       | Threshold matrix decides pixel ON/OFF | Preserved  |
| <b>Error Diffusion</b> | Spreads rounding error to neighbors   | Preserved  |

**Dithering rule:** Pixel ON if  $\text{ImageIntensity}(x,y) > D_n(i,j)$ , where  $i = x \bmod n$ ,  $j = y \bmod n$

**Error Diffusion (Floyd-Steinberg):** Distributes quantization error to unprocessed neighbors → smooth gradients without banding.

---

#### Image Files

- Structure: **Header** (format, dimensions) + **Image Data**
- Compression: **Run-Length Encoding (RLE)** — e.g.,  $\text{AAAAAABBBCC} \rightarrow (A,6), (B,3), (C,2)$

---

#### Exam Math

##### Q1a: What is quantization?

Quantization is the process of mapping a continuous range of values into a smaller, finite, discrete set. In image representation, it means converting continuous light intensities into discrete digital values — for example, mapping analog color intensity to an 8-bit value in the range 0–255.

---

##### Q1b: Maximum memory for 10 images using direct coding (640×520 pixels, 3 bytes/pixel)?

###### Given:

- Image dimensions: 640 × 520 pixels
- Color depth: 3 bytes per pixel (1 byte each for R, G, B)
- Number of images: 10

###### Step 1 — Total pixels per image:

$$640 \times 520 = 332,800 \text{ pixels}$$

###### Step 2 — Memory per image:

$$332,800 \text{ pixels} \times 3 \text{ bytes/pixel} = 998,400 \text{ bytes}$$

**Step 3 — Total memory for 10 images:**

$$998400 \times 10 = 9984000 \text{ bytes}$$

$$\text{Converting: } 9984000 \div 1024 = 9750 \text{ KB} \approx 9.52 \text{ MB}$$

---

**Q1c: How many colors can be stored in a lookup table if each image's index memory must not exceed 500 KB?**

**Given:**

- Image size:  $640 \times 520 = 332,800$  pixels (from Q1b)
- Memory limit per image: 500 KB
- Goal: find maximum number of bits per pixel for the index, then find the number of colors

**Step 1 — Convert 500 KB to bytes:**

$$500 \times 1024 = 512000 \text{ bytes}$$

**Step 2 — Convert bytes to bits:**

$$512000 \times 8 = 4096000 \text{ bits}$$

**Step 3 — Calculate available bits per pixel:**

$$\frac{4096000 \text{ bits}}{332800 \text{ pixels}} \approx 12.307 \text{ bits/pixel}$$

**Step 4 — Round down to a whole number (must be integer bit depth):**

$$\lfloor 12.307 \rfloor = 12 \text{ bits/pixel}$$

*(We round down, not up, to stay within the 500 KB limit.)*

**Step 5 — Calculate number of representable colors:**

$$2^{12} = 4096 \text{ colors}$$

---

**Bonus Q: If 2-byte pixel values are used in a 24-bit lookup table, how many bytes does the lookup table occupy?**

**Given:**

- Pixel size: 2 bytes = 16 bits → used as an index
- Each LUT entry: 24-bit color = 3 bytes

**Step 1 — Number of LUT entries (one per possible index value):**

$$2^{16} = 65536 \text{ entries}$$

**Step 2 — Size of each entry:**

$$24 \text{ bits} \div 8 = 3 \text{ bytes per entry}$$

**Step 3 — Total LUT size:**

$$65536 \times 3 = 196608 \text{ bytes}$$

---

## Chapter 3: Scan Conversion

### Scan-Converting a Point

$$X_{\text{pixel}} = \lfloor x + 0.5 \rfloor, \quad Y_{\text{pixel}} = \lfloor y + 0.5 \rfloor$$

**Example:** Map continuous point  $(10.3, 20.7)$  to a pixel.

**Step 1 — Add 0.5 to each coordinate:**

$$x + 0.5 = 10.3 + 0.5 = 10.8$$

$$y + 0.5 = 20.7 + 0.5 = 21.2$$

**Step 2 — Apply floor:**

$$X_{\text{pixel}} = \lfloor 10.8 \rfloor = 10, \quad Y_{\text{pixel}} = \lfloor 21.2 \rfloor = 21$$

**Result:** Pixel plotted at  $(10, 21)$

---

### Line Drawing Algorithms

#### 1. Slope-Intercept Method ( $y = mx + b$ )

**Steps:**

1. Compute slope:  $m = \frac{y_2 - y_1}{x_2 - x_1}$
2. Compute y-intercept:  $b = y_1 - m \cdot x_1$
3. For each integer  $x$  from  $x_1$  to  $x_2$ : compute  $y = mx + b$ , round to nearest integer, plot  $(x, \text{round}(y))$

**Full Worked Example — Line from (2, 3) to (6, 5):**

**Step 1 — Calculate slope:**

$$m = \frac{5 - 3}{6 - 2} = \frac{2}{4} = 0.5$$

**Step 2 — Calculate y-intercept:**

$$b = y_1 - m \cdot x_1 = 3 - (0.5 \cdot 2) = 3 - 1 = 2$$

**Step 3 — Iterate and compute y for each x (using  $y = 0.5x + 2$ ):**

| $x$ | Calculation            | $y$ (float) | $\text{round}(y)$ | Plot   |
|-----|------------------------|-------------|-------------------|--------|
| 2   | $0.5(2) + 2 = 1 + 2$   | 3.0         | 3                 | (2, 3) |
| 3   | $0.5(3) + 2 = 1.5 + 2$ | 3.5         | 4                 | (3, 4) |
| 4   | $0.5(4) + 2 = 2 + 2$   | 4.0         | 4                 | (4, 4) |
| 5   | $0.5(5) + 2 = 2.5 + 2$ | 4.5         | 5                 | (5, 5) |
| 6   | $0.5(6) + 2 = 3 + 2$   | 5.0         | 5                 | (6, 5) |



Advantage



Disadvantage

✔

Advantage

✖

Disadvantage

|                      |                                           |
|----------------------|-------------------------------------------|
| Simple to understand | Floating-point multiplication every pixel |
| Direct calculation   | Visual gaps when $m > 1$                  |

## 2. DDA (Digital Differential Analyzer)

### Steps:

1. Compute differences:  $dx = x_2 - x_1$ ,  $dy = y_2 - y_1$
2. Determine step count:  $\text{step} = \max(|dx|, |dy|)$  — ensures no gaps regardless of slope
3. Compute per-step increments:  $x_{\text{inc}} = dx / \text{step}$ ,  $y_{\text{inc}} = dy / \text{step}$
4. Initialize:  $x = x_1$ ,  $y = y_1$
5. For each iteration: plot  $(\text{round}(x), \text{round}(y))$ , then  $x = x + x_{\text{inc}}$ ,  $y = y + y_{\text{inc}}$

### Full Worked Example — Line from (2, 3) to (6, 5):

#### Step 1 — Compute differences:

$$dx = 6 - 2 = 4, \quad dy = 5 - 3 = 2$$

#### Step 2 — Determine step count:

$$\text{step} = \max(|4|, |2|) = \max(4, 2) = 4$$

#### Step 3 — Compute increments:

$$x_{\text{inc}} = \frac{4}{4} = 1.0, \quad y_{\text{inc}} = \frac{2}{4} = 0.5$$

#### Step 4 — Initialize: $x = 2.0, y = 3.0$

#### Step 5 — Iterate (at each step: plot → add increments):

| Iter | $x$ | $y$ | $\text{round}(x)$ | $\text{round}(y)$ | Plot  | $x_{\text{next}}$ | $y_{\text{next}}$ |
|------|-----|-----|-------------------|-------------------|-------|-------------------|-------------------|
| 0    | 2.0 | 3.0 | 2                 | 3                 | (2,3) | $2.0+1.0=3.0$     | $3.0+0.5=3.5$     |
| 1    | 3.0 | 3.5 | 3                 | 4                 | (3,4) | $3.0+1.0=4.0$     | $3.5+0.5=4.0$     |
| 2    | 4.0 | 4.0 | 4                 | 4                 | (4,4) | $4.0+1.0=5.0$     | $4.0+0.5=4.5$     |
| 3    | 5.0 | 4.5 | 5                 | 5                 | (5,5) | $5.0+1.0=6.0$     | $4.5+0.5=5.0$     |
| 4    | 6.0 | 5.0 | 6                 | 5                 | (6,5) | —                 | —                 |

✔

Advantage

✖

Disadvantage

|                   |                                         |
|-------------------|-----------------------------------------|
| No multiplication | Still uses floating-point               |
| No gaps in line   | Cumulative rounding error on long lines |

## 3. Bresenham's Line Algorithm (Integer Only)

### Steps (for $m < 1$ ):

1. Compute:  $dx = x_2 - x_1$ ,  $dy = y_2 - y_1$
2. Pre-compute constants:  $2dy$  and  $2dy - 2dx$
3. Initialize:  $x = x_1$ ,  $y = y_1$ ,  $P_0 = 2dy - dx$
4. At each step: plot  $(x, y)$ , increment  $x = x + 1$ , then:
  - If  $P_n < 0$ :  $y$  stays,  $P_{n+1} = P_n + 2dy$
  - If  $P_n \geq 0$ :  $y = y + 1$ ,  $P_{n+1} = P_n + 2dy - 2dx$
5. Repeat until  $x = x_2$

### Full Worked Example — Line from (2, 3) to (6, 5):

#### Step 1 — Compute differences:

$$dx = 6 - 2 = 4, \quad dy = 5 - 3 = 2$$

#### Step 2 — Pre-compute constants:

$$2dy = 2 \times 2 = 4$$

$$2dy - 2dx = 4 - 2(4) = 4 - 8 = -4$$

#### Step 3 — Initial decision variable:

$$P_0 = 2dy - dx = 4 - 4 = 0$$

#### Step 4 — Initialize: plot (2, 3), then iterate:

| Step | Current $(x,y)$ | $P_n$ | $P_n < 0$ ?     | Decision       | Next $(x,y)$ | $P_{n+1}$ calculation | $P_{n+1}$ |
|------|-----------------|-------|-----------------|----------------|--------------|-----------------------|-----------|
| Init | (2, 3)          | —     | —               | —              | —            | $P_0 = 0$             | 0         |
| 1    | (2,3) plotted   | 0     | No ( $\geq 0$ ) | $y$ increments | (3, 4)       | $0 + (-4)$            | -4        |
| 2    | (3,4) plotted   | -4    | Yes             | $y$ stays      | (4, 4)       | $-4 + 4$              | 0         |
| 3    | (4,4) plotted   | 0     | No ( $\geq 0$ ) | $y$ increments | (5, 5)       | $0 + (-4)$            | -4        |
| 4    | (5,5) plotted   | -4    | Yes             | $y$ stays      | (6, 5)       | end                   | —         |

Final pixels plotted: (2,3), (3,4), (4,4), (5,5), (6,5)



Advantage



Disadvantage

Integer-only (fastest)

More complex to implement for all slopes

No rounding needed

Different cases needed for  $m > 1$ , negatives

## Circle & Ellipse Drawing

### 8-Way Symmetry (Circles)

Compute one octant ( $x \leq y$ ), mirror to get 8 points:

| Octant | Transform  | Pixel (from point (2,5)) |
|--------|------------|--------------------------|
| 1      | $(x, y)$   | (2, 5)                   |
| 2      | $(y, x)$   | (5, 2)                   |
| 3      | $(-y, x)$  | (-5, 2)                  |
| 4      | $(-x, y)$  | (-2, 5)                  |
| 5      | $(-x, -y)$ | (-2, -5)                 |
| 6      | $(-y, -x)$ | (-5, -2)                 |
| 7      | $(y, -x)$  | (5, -2)                  |
| 8      | $(x, -y)$  | (2, -5)                  |

Ellipses use **4-way symmetry** only —  $x$  and  $y$  cannot be swapped.

**Performance gain:** 87.5% fewer calculations for circles; 75% for ellipses.

### Bresenham's Circle Algorithm

**Initialize:**  $x = 0$ ,  $y = r$ ,  $d_0 = 3 - 2r$

**At each step (while  $x \leq y$ ):**

- Plot 8 symmetric points for current  $(x, y)$
- If  $d < 0$ :  $y$  stays,  $d = d + 4x + 6$
- If  $d \geq 0$ :  $y = y - 1$ ,  $d = d + 4x - 4y + 10$
- Always:  $x = x + 1$

**Full Worked Example — Circle with  $r = 5$ , centered at  $(0,0)$ :**

**Step 1 — Initialize:**

$x = 0$ ,  $y = 5$

$d_0 = 3 - 2(5) = 3 - 10 = -7$

**Step 2 — Iterate** (note:  $d$  update uses  $x$  and  $y$  values *before* incrementing):

| Step | $d_n$ | $d < 0$ ? | Plot $(x,y)$ | $d_{n+1}$ calculation        | $d_{n+1}$ | New $(x,y)$ |
|------|-------|-----------|--------------|------------------------------|-----------|-------------|
| Init | —     | —         | (0, 5)       | $d_0 = -7$                   | -7        | —           |
| 1    | -7    | Yes       | (1, 5)       | $-7 + 4(0) + 6 = -7 + 0 + 6$ | -1        | (1, 5)      |



| Step | $x_n$ | $x < 0$ ? | Plot $(x, y)$ | $x_{n+1}$ calculation                     | $x_{n+1}$ | New $(x, y)$ |
|------|-------|-----------|---------------|-------------------------------------------|-----------|--------------|
| 2    | -1    | Yes       | (2, 5)        | $-1 + 4(1) + 6 = -1 + 4 + 6$              | 9         | (2, 5)       |
| 3    | 9     | No        | (3, 4)        | $9 + 4(2) - 4(5) + 10 = 9 + 8 - 20 + 10$  | 7         | (3, 4)       |
| 4    | 7     | No        | (4, 3)        | $7 + 4(3) - 4(4) + 10 = 7 + 12 - 16 + 10$ | 13        | (4, 3)       |

Stop: next  $x = 5 > y = 3$ , so first octant is complete.

**Points computed**  $(x, y)$ :  $(0, 5)$ ,  $(1, 5)$ ,  $(2, 5)$ ,  $(3, 4)$ ,  $(4, 3)$  — each is mirrored using 8-way symmetry to produce the full circle.

### Midpoint Circle Algorithm

**Initialize:**  $x = 0$ ,  $y = r$ ,  $p_0 = 1 - r$

**At each step (while  $x \leq y$ ):**

- Plot 8 symmetric points for current  $(x, y)$
- If  $p < 0$ :  $y$  stays,  $p = p + 2x + 3$
- If  $p \geq 0$ :  $y = y - 1$ ,  $p = p + 2x - 2y + 5$
- Always:  $x = x + 1$

**Full Worked Example — Circle with  $r = 5$ , centered at  $(0, 0)$ :**

**Step 1 — Initialize:**

$x = 0$ ,  $y = 5$

$p_0 = 1 - r = 1 - 5 = -4$

**Step 2 — Iterate** (note:  $p$  update uses  $x$  and  $y$  values *before* incrementing  $x$ , but *after* decrementing  $y$  when applicable):

| Step | $p_n$ | $p < 0$ ? | Plot $(x, y)$ | $p_{n+1}$ calculation                  | $p_{n+1}$ | New $(x, y)$ |
|------|-------|-----------|---------------|----------------------------------------|-----------|--------------|
| Init | —     | —         | (0, 5)        | $p_0 = -4$                             | -4        | —            |
| 1    | -4    | Yes       | (1, 5)        | $-4 + 2(0) + 3 = -4 + 0 + 3$           | -1        | (1, 5)       |
| 2    | -1    | Yes       | (2, 5)        | $-1 + 2(1) + 3 = -1 + 2 + 3$           | 4         | (2, 5)       |
| 3    | 4     | No        | (3, 4)        | $4 + 2(2) - 2(5) + 5 = 4 + 4 - 10 + 5$ | 3         | (3, 4)       |
| 4    | 3     | No        | (4, 3)        | $3 + 2(3) - 2(4) + 5 = 3 + 6 - 8 + 5$  | 6         | (4, 3)       |

Stop: next  $x = 5 > y = 3$ , first octant complete.

**Points computed:**  $(0,5), (1,5), (2,5), (3,4), (4,3)$  — mirrored using 8-way symmetry.

Midpoint is more adaptable than Bresenham's Circle — it is the standard method used for ellipses. For ellipses, split the curve into two regions at the point where slope  $= -1$ .

---

## Scan-Converting Characters (Fonts)

### Bitmap (Raster) Fonts

- Store characters as pre-calculated binary grids
- Rendering = copy grid to screen memory (BitBlt)



#### Advantage



#### Disadvantage

Extremely fast

Blocky when scaled

Pixel-perfect at native size

Separate file per size needed

### Outline (Vector) Fonts (e.g., TrueType, OpenType)

- Store mathematical curves (Bézier) and lines
- Rendering pipeline: **Fetch geometry** → **Transform** → **Rasterize boundary** → **Scanline fill** → **Hint**



#### Advantage



#### Disadvantage

Infinitely scalable

Computationally expensive

| Single file for all sizes | Blurry at small sizes without hinting |

---

## Scan-Converting Arcs, Sectors, Rectangles

### Arcs

Three approaches:

1. **Trigonometric:**  $x = h + r \cos(\theta)$ ,  $y = k + r \sin(\theta)$  — accurate but slow (uses sin/cos)
2. **Polynomial:**  $y = \sqrt{r^2 - x^2}$  — uses square roots, no symmetry
3. **Bresenham (avoid for arcs):** Must compute entire circle then check bounds — inefficient, risks infinite loop

### Sectors (Pie Slice)

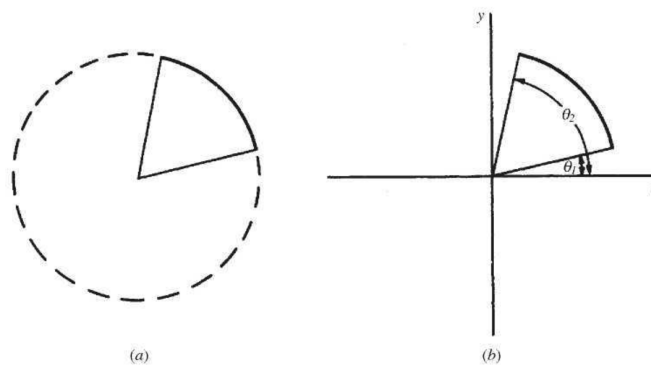


Fig. 3-13

**Fig. 3-13** Scan-converting a sector (pie slice): (a) shows the arc and sector, (b) shows the angles and endpoints used for filling.

1. Draw arc from  $\theta_1$  to  $\theta_2$
2. Calculate endpoints:  $(h + r \cos \theta_1, k + r \sin \theta_1)$  and  $(h + r \cos \theta_2, k + r \sin \theta_2)$
3. Draw two lines from center to each endpoint (use Bresenham's line)

### Rectangle (Axis-aligned)

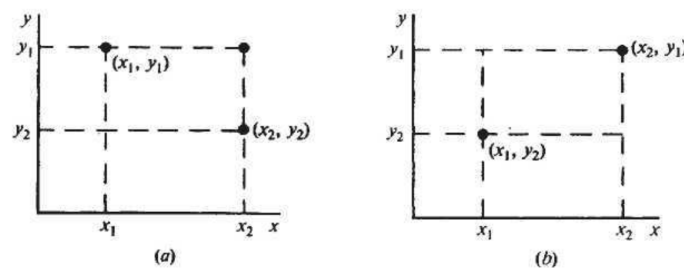


Fig. 3-16

**Fig. 3-16** Scan-converting a rectangle: (a) and (b) show how the four corners are used to define the rectangle for filling or drawing edges.

- Input: two opposite corners  $(x_1, y_1)$  and  $(x_2, y_2)$
- Derive:  $(x_1, y_2)$  and  $(x_2, y_1)$
- Draw 4 edges using any line algorithm

### Region Filling

**Interior-defined region:** A region where filling begins from a seed point inside the boundary and continues outward until the boundary is reached.

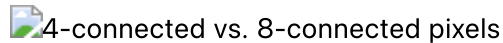
**Boundary-defined region:** A region enclosed by a specific boundary color, where filling begins from a seed point inside the area and continues outward pixel-by-pixel until the specific boundary color is reached.



**Fig. 3-17** (a) Boundary-defined region: filling is constrained by a boundary color. (b) Interior-defined region: filling spreads outward from a seed point until the boundary is reached.

## Pixel Connectivity

| Type        | Neighbors                                     |
|-------------|-----------------------------------------------|
| 4-Connected | Up, Down, Left, Right only                    |
| 8-Connected | All 8 surrounding pixels (includes diagonals) |



**Fig. 3-18** (a) 4-connected neighbors: only up, down, left, right. (b) 8-connected neighbors: includes diagonals. (c) Example showing how connectivity affects region filling.

**Critical rule:** If boundary is 8-connected → fill must be 4-connected, and vice versa (prevents leaking).

### 1. Boundary-Fill (Recursive)

Used when region is enclosed by a specific **boundary color**.

#### Steps:

1. Start at seed pixel
2. If pixel = fill color OR boundary color → stop
3. Else: paint pixel with fill color, recurse on neighbors

| ✓ Advantage | ✗ Disadvantage |
|-------------|----------------|
|-------------|----------------|

|                           |                                    |
|---------------------------|------------------------------------|
| Works on irregular shapes | Stack overflow on large regions    |
| Easy to implement         | Fails if boundary is multi-colored |

### 2. Flood-Fill (Recursive)

Used when region is defined by a uniform **interior color** (e.g., paint bucket tool).

#### Steps:

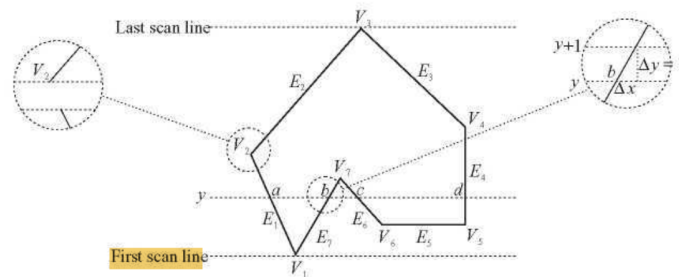
1. Start at seed, record **Old\_Color**
2. If pixel ≠ **Old\_Color** OR already = **New\_Color** → stop
3. Else: paint with **New\_Color**, recurse on neighbors

| ✓ Advantage | ✗ Disadvantage |
|-------------|----------------|
|-------------|----------------|

|                            |                       |
|----------------------------|-----------------------|
| No boundary color needed   | Stack overflow risk   |
| Natural stopping condition | Cannot fill gradients |

### 3. Scanline Fill (Optimized Polygon Fill)

We represent a polygonal region in terms of a sequence of vertices  $V_1, V_2, V_3, \dots$ , that are connected by edges  $E_1, E_2, E_3, \dots$ , (see Fig. 3-19). We assume that each vertex  $V_i$  has already been scan-converted to integer coordinates  $(x_i, y_i)$ .



**Fig. 3-19** Scan-converting a polygonal region.

**Fig. 3-19** Scan-converting a polygonal region.

**Table 3-1** An edge list.

| Edge  | $y_{\min}$ | $y_{\max}$ | $x$ coordinate of vertex with $y = y_{\min}$ | $1/m$   |
|-------|------------|------------|----------------------------------------------|---------|
| $E_1$ | $y_1$      | $y_2 - 1$  | $x_1$                                        | $1/m_1$ |
| $E_7$ | $y_1$      | $y_7$      | $x_1$                                        | $1/m_7$ |
| $E_4$ | $y_5$      | $y_4 - 1$  | $x_5$                                        | $1/m_4$ |
| $E_6$ | $y_6$      | $y_7$      | $x_6$                                        | $1/m_6$ |
| $E_2$ | $y_2$      | $y_3$      | $x_2$                                        | $1/m_2$ |
| $E_3$ | $y_4$      | $y_3$      | $x_4$                                        | $1/m_3$ |

**Table 3-1** An edge list for scanline fill.

Fills row-by-row using the polygon's geometric edge data. Eliminates recursion.

### Core Concepts:

| Concept                     | Explanation                                                                                                                                    |
|-----------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Sweeping Scanline</b>    | Process one horizontal line at a time, bottom to top                                                                                           |
| <b>Vertex Problem</b>       | Fix: shorten lower edge by 1 ( $y_{\max} - 1$ ) at monotonic vertices                                                                          |
| <b>Horizontal Rule</b>      | Ignore horizontal edges                                                                                                                        |
| <b>Edge Activation Rule</b> | Edge is active from $y_{\min}$ up to (but not including) $y_{\max}$ ; it is deactivated/removed after the scanline at $y_{\max}$ is processed. |
| <b>Odd-Even Rule</b>        | Fill pixels between paired intersection points                                                                                                 |
| <b>Edge Coherence</b>       | $x_{\text{new}} = x_{\text{old}} + \frac{1}{m}$ — no recalculation per row                                                                     |

## Algorithm Steps:

1. **Build Edge Table (ET):** For each non-horizontal edge compute  $y_{\min}$ ,  $y_{\max}$ ,  $x$ ,  $\frac{1}{m}$ ; apply  $y_{\max}-1$  rule; sort by  $y_{\min}$
2. **Initialize:**  $y = \min(y_{\min})$
3. **Update AEL:** Move edges with  $y_{\min} = y$  into Active Edge List
4. **Sort AEL** by  $x$  (left to right)
5. **Fill:** Pair edges (0&1, 2&3, ...); fill between each pair
6. **Remove** edges where  $y_{\max} = y$
7. **Increment:**  $y = y + 1$ ; update  $x_{\text{new}} = x + \frac{1}{m}$  for remaining AEL edges
8. **Repeat** until ET and AEL are empty

**Worked Example — Q2(b):** Vertices  $(-4,0), (0,2), (4,0), (2,4), (6,4), (0,8), (-6,4), (-4,4)$

### Step 1 — Form edges and discard horizontal ones:

| Edge | From → To                   | Horizontal?          |
|------|-----------------------------|----------------------|
| E1   | $(-4,0) \rightarrow (0,2)$  | No — keep            |
| E2   | $(0,2) \rightarrow (4,0)$   | No — keep            |
| E3   | $(4,0) \rightarrow (2,4)$   | No — keep            |
| E4   | $(2,4) \rightarrow (6,4)$   | <b>Yes — discard</b> |
| E5   | $(6,4) \rightarrow (0,8)$   | No — keep            |
| E6   | $(0,8) \rightarrow (-6,4)$  | No — keep            |
| E7   | $(-6,4) \rightarrow (-4,4)$ | <b>Yes — discard</b> |
| E8   | $(-4,4) \rightarrow (-4,0)$ | No — keep            |

### Step 2 — Compute ET values using $\frac{1}{m} = \frac{x_2 - x_1}{y_2 - y_1}$ :

- **E1**  $(-4,0) \rightarrow (0,2)$ :  $y_{\min}=0, y_{\max}=2, x=-4, \frac{1}{m} = \frac{0-(-4)}{2-0} = \frac{4}{2} = 2$
- **E2**  $(0,2) \rightarrow (4,0)$ :  $y_{\min}=0, y_{\max}=2, x=4, \frac{1}{m} = \frac{4-0}{0-2} = \frac{4}{-2} = -2$
- **E3**  $(4,0) \rightarrow (2,4)$ :  $y_{\min}=0, y_{\max}=4, x=4, \frac{1}{m} = \frac{2-4}{4-0} = \frac{-2}{4} = -0.5$
- **E5**  $(6,4) \rightarrow (0,8)$ :  $y_{\min}=4, y_{\max}=8, x=6, \frac{1}{m} = \frac{0-6}{8-4} = \frac{-6}{4} = -1.5$
- **E6**  $(0,8) \rightarrow (-6,4)$ :  $y_{\min}=4, y_{\max}=8, x=-6, \frac{1}{m} = \frac{-6-0}{4-8} = \frac{-6}{-4} = 1.5$
- **E8**  $(-4,4) \rightarrow (-4,0)$ :  $y_{\min}=0, y_{\max}=4, x=-4, \frac{1}{m} = \frac{-4-(-4)}{0-4} = \frac{0}{-4} = 0$

### Step 3 — Final Edge Table (sorted by $y_{\min}$ ):

| Edge | $y_{\min}$ | $y_{\max}$ | $x$ at $y_{\min}$ | $\frac{1}{m}$ |
|------|------------|------------|-------------------|---------------|
| E1   | 0          | 2          | -4                | 2             |
| E2   | 0          | 2          | 4                 | -2            |

| Edge | $y_{\min}$ | $y_{\max}$ | $x$ at $y_{\min}$ | $1/m$ |
|------|------------|------------|-------------------|-------|
| E3   | 0          | 4          | 4                 | -0.5  |
| E8   | 0          | 4          | -4                | 0     |
| E5   | 4          | 8          | 6                 | -1.5  |
| E6   | 4          | 8          | -6                | 1.5   |

**Step 4 — Initialize:**  $y = 0$  (smallest  $y_{\min}$ )

**Scanline  $y = 0$ :**

- AEL: add E1, E2, E3, E8 (all have  $y_{\min} = 0$ )
- Sort AEL by  $x$ :  $-4$  (E8),  $-4$  (E1),  $4$  (E3),  $4$  (E2)
- Pair (E8,E1)  $\rightarrow$  fill from  $-4$  to  $-4$   $\rightarrow$  zero width, **no fill**
- Pair (E3,E2)  $\rightarrow$  fill from  $4$  to  $4$   $\rightarrow$  zero width, **no fill**
- No edges have  $y_{\max} = 0$ , so none are removed
- Update  $x$  values:  $x_{\text{new}} = x_{\text{old}} + \frac{1}{m}$

| Edge | Old $x$ | $1/m$ | New $x$            |
|------|---------|-------|--------------------|
| E1   | -4      | 2     | $-4 + 2 = -2$      |
| E2   | 4       | -2    | $4 + (-2) = 2$     |
| E3   | 4       | -0.5  | $4 + (-0.5) = 3.5$ |
| E8   | -4      | 0     | $-4 + 0 = -4$      |

**Scanline  $y = 1$ :**

- No new edges enter AEL
- AEL sorted by updated  $x$ :  $-4$  (E8),  $-2$  (E1),  $2$  (E2),  $3.5$  (E3)
- Pair (E8, E1)  $\rightarrow$  **fill from  $x = -4$  to  $x = -2$**  ✓
- Pair (E2, E3)  $\rightarrow$  **fill from  $x = 2$  to  $x = 3.5$**  ✓
- Update  $x$  values:

| Edge | Old $x$ | $1/m$ | New $x$         |
|------|---------|-------|-----------------|
| E1   | -2      | 2     | $-2 + 2 = 0$    |
| E2   | 2       | -2    | $2 + (-2) = 0$  |
| E3   | 3.5     | -0.5  | $3.5 - 0.5 = 3$ |
| E8   | -4      | 0     | $-4 + 0 = -4$   |

**Scanline  $y = 2$ :**

- Remove E1 and E2 ( $y_{\max} = 2 = y$ )

- AEL now: E3, E8 (sorted:  $-4, 3$ )
- **Fill from  $x = -4$  to  $x = 3$  ✓**
- Update:

| Edge | Old $x$ | $\Delta x$ | New $x$         |
|------|---------|------------|-----------------|
| E3   | 3       | $-0.5$     | $3 - 0.5 = 2.5$ |
| E8   | $-4$    | 0          | $-4$            |

#### Scanline $y = 3$ :

- AEL: E3, E8 (sorted:  $-4, 2.5$ )
- **Fill from  $x = -4$  to  $x = 2.5$  ✓**
- Update: E3  $\rightarrow 2.5 - 0.5 = 2$ ; E8  $\rightarrow -4$

#### Scanline $y = 4$ :

- Remove E3 and E8 ( $y_{\max} = 4 = y$ )
- Add E5 and E6 from ET ( $y_{\min} = 4$ )
- AEL: E5 ( $x=6$ ), E6 ( $x=-6$ )  $\rightarrow$  sorted:  $-6, 6$
- **Fill from  $x = -6$  to  $x = 6$  ✓**
- Update: E5  $\rightarrow 6 - 1.5 = 4.5$ ; E6  $\rightarrow -6 + 1.5 = -4.5$

(Continue similarly for  $y = 5, 6, 7$  until  $y = 8$  where E5 and E6 are removed and both ET and AEL are empty.)



#### Advantage



#### Disadvantage

No recursion

ET and AEL maintenance overhead

Fast via edge coherence

AEL re-sort needed when edges cross

Handles concave polygons & holes

#### Common Mistakes:

- Including horizontal edges in ET
- Forgetting to sort AEL by  $x$  each scanline
- Calculating  $\frac{1}{m}$  as  $\frac{\Delta y}{\Delta x}$  instead of  $\frac{\Delta x}{\Delta y}$
- Forgetting  $y_{\max} - 1$  rule at monotonic vertices

## Anti-Aliasing

### Aliasing Artifacts

- Visual distortions caused by approximating continuous objects with discrete pixels.

| Artifact | Description |
|----------|-------------|
|----------|-------------|



| Artifact                    | Description                                                                                                   |
|-----------------------------|---------------------------------------------------------------------------------------------------------------|
| <b>Staircase (Jaggies)</b>  | Jagged edges on diagonal lines/curves                                                                         |
| <b>Unequal Brightness</b>   | Diagonals appear dimmer than horizontal/vertical lines. Because distance between two diagonal pixel is larger |
| <b>Picket Fence Problem</b> | When an object does not fit into, the pixel grid properly                                                     |
| <b>Edge Flickering</b>      | Edge vibrate during animation                                                                                 |
| <b>Moire Pattern</b>        | Interference occur when high frequency texture are sampled                                                    |

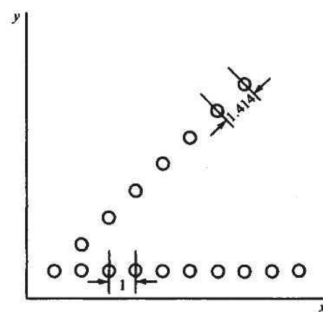
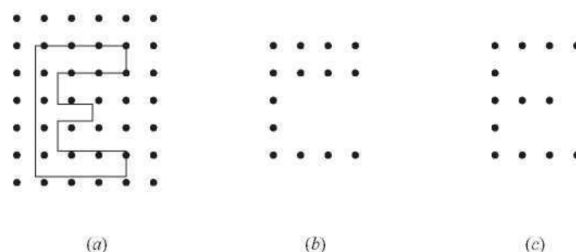


Fig. 3-23

**Fig. Unequal Brightness**



**Fig. 3-25** Scan-converting an outline font.

**Fig. Picket Fence Problem with the outline font.**

Suppose we want to scan-convert the uppercase character "E" in (a) from its outline description to a bitmap consisting of pixels inside the region defined by the outline. The result in (b) exhibits both asymmetry (the upper arm of the character is twice as thick as the other parts) and dropout (the middle arm is absent). A slight adjustment and/or realignment of the outline can lead to a reasonable outcome (c).

## . Why Anti-Aliasing is Needed

- Simply increasing resolution reduces jaggedness but **requires a lot of memory and processing**.
- Anti-aliasing **smooths edges without increasing resolution**, making images appear natural.

## . Main Anti-Aliasing Techniques with Examples

### A. Pre-Filtering

- Works **before converting to pixels**.
- Each pixel's color is set **based on how much of it is covered by the object**.

#### Example:

- Pixel partially covered by a white line:

```
Coverage = 50% (0.5)
Object color = White (1)
Background color = Black (0)
Pixel color =  $1 \times 0.5 + 0 \times 0.5 = 0.5 \rightarrow \text{gray}$ 
```

- Fully inside pixel  $\rightarrow$  full white, outside  $\rightarrow$  black.

**Result:** Smooth edge; shape remains sharp in the center.

---

### B. Supersampling (Post-Filtering)

- Each pixel is divided into **subpixels** (e.g.,  $3 \times 3$ ).
- Count subpixels inside the object  $\rightarrow$  average their colors  $\rightarrow$  final pixel color.

#### Example:

- Pixel divided into  $3 \times 3 = 9$  subpixels
- 6 subpixels are inside a line  $\rightarrow$  coverage =  $6/9 \approx 0.67$
- Object color = white (1), background = black (0)
- Pixel color =  $1 \times 0.67 + 0 \times 0.33 = 0.67 \rightarrow \text{gray}$

#### Visual Idea:

```
[• • •]
[• o o]
[o o o]
```

- • = subpixel inside object
- o = subpixel outside object
- Coverage =  $6/9 \rightarrow$  pixel color 0.67 (blended)

**Result:** Smooth diagonal lines and edges.

---

### C. Pixel Phasing

- Hardware-based: shifts pixels **slightly from their normal positions** to align with true line/contour.
- Reduces stair-step effect without blurring edges.

**Example:**

- A diagonal line across a pixel grid may look jagged.
- Pixel positions are shifted **fractionally toward the line**, so the line looks smoother.

**Visual Idea (simplified):**

| Before: | After: |
|---------|--------|
| • 0     | • •    |
| 0 •     | 0 •    |

- Line edges appear aligned → stair-step effect minimized.

## D. Gray-Level / Color Blending

- Uses **intermediate shades or colors** between object and background.
- Only edge pixels are blended, making transitions smooth.

**Example:**

- White line on black background
- Edge pixel = 50% covered → gray
- Edge pixel = 25% covered → dark gray

**Visual Idea:**

Background → Dark Gray → Gray → White (center of line)

**Result:** Human eye sees a smooth, continuous line.

**In short:**

Anti-aliasing tricks your eyes by **blending colors, averaging sub-pixels, or shifting pixels slightly**, so edges appear **smooth and natural**, even on low-resolution screens.